CS-1204
Object-Oriented Programming
(3+1)

Week 02

By

Engr. Muhammad Awais

Lectures : April 21 & April 22 , 2025
Lab: April 25, 2025

Contents

- Constructors
- Why use Constructors
- Types of Constructors
- Constructor Overloading
- Destructors

What is a Constructor?

- A constructor is a special function in a class that is automatically called when an object is created.
- **Key Points:**
  - Same name as the class.
  - No return type (not even void).
  - Automatically invoked

Why Use Constructors?

- **Problem with Setters:**
  - Must call setters manually.
  - Risk of forgetting to initialize values.
  - Code becomes error-prone and less readable.
- **Solution:**
  - Constructors automatically initialize object data.
  - Ensure object is always in a valid state.

Without Constructor (Manual Initialization)

```
class Student {
    string name;
    int age;
public:
    void setName(string n) { name = n; }
    void setAge(int a) { age = a; }
    void display() { cout << name << ", " << age << endl; }
};

int main() {
    Student s;
    s.setName("Ali");
    s.setAge(20);
    s.display();
}
```

Risk: Forgetting to set data

With Constructor (Automatic Initialization)

```
class Student {
    string name;
    int age;
public:
    Student(string n, int a) {
        name = n;
        age = a;
    }
    void display() { cout << name << ", " << age << endl; }
};

int main() {
    Student s("Ali", 20);
    s.display();
}
```

Cleaner, safer, and faster!

When to Use a Constructor?

- **Use a Constructor When:**
  - You need to always initialize data.
  - You want clean and safe code.
  - You want to create many objects efficiently.

- **Use Setters When:**
  - Values are not known at creation time.
  - You need to update values later

Types of Constructors?

- Default Constructor
- Parametrized Constructor
- Copy Constructor

Default Constructor

- A constructor that takes no arguments is called a default constructor.
- It's either defined by the programmer or automatically provided by the compiler if no other constructor is defined.

```cpp
#include <iostream>
using namespace std;

class Student {
private:
    int rollNo;
    string name;

public:
    // Default Constructor
    Student() {
        rollNo = 0;
        name = "Unknown";
    }

    void display() {
        cout << "Roll No: " << rollNo << ", Name: " << name << endl;
    }
};

int main() {
    Student s1;  // Calls default constructor
    s1.display();

    return 0;
}
```

When to Use:

When you want to provide default values during object creation.

Parametrized Constructor

- A constructor that takes parameters is called a parameterized constructor.
- It allows you to initialize objects with custom values.

```cpp
#include <iostream>
using namespace std;

class Student {
private:
    int rollNo;
    string name;

public:
    // Parameterized Constructor
    Student(int r, string n) {
        rollNo = r;
        name = n;
    }

    void display() {
        cout << "Roll No: " << rollNo << ", Name: " << name << endl;
    }
};

int main() {
    Student s1(101, "Amina");  // Calls parameterized constructor
    s1.display();

    return 0;
}
```

When to Use:

When you want to initialize objects with specific values provided at runtime or object creation.

Info : Compiler-Provided Constructors

- If no constructor is explicitly defined:
  - C++ automatically provides a default constructor.
  - If you define a parameterized constructor but not a default one, the compiler does not provide a default constructor.

```cpp
class Person {
private:
    string name;
public:
    // Parameterized Constructor
    Person(string n) {
        name = n;
    }
};

int main() {
    Person p1("Ali");  // ✅ Works
    Person p2;         // ❌ Error: No default constructor
}
```

Why Error:

Because once you define any constructor, the compiler does not generate the default constructor for you. If you still need one, you must write it yourself.

```cpp
class Person {
private:
    string name;

public:
    // Default Constructor
    Person() {
        name = "Unknown";
    }

    // Parameterized Constructor
    Person(string n) {
        name = n;
    }
};

int main() {
    Person p1("Ali");  // ✅ Works
    Person p2;         // ✅ Now works too

    return 0;
}
```

Constructor Overloading

- You can define multiple constructors with different parameters (constructor overloading):

```
class Student {
private:
    int rollNo;
    string name;

public:
    // Default Constructor
    Student() {
        rollNo = 0;
        name = "Unknown";
    }

    // Parameterized Constructor
    Student(int r, string n) {
        rollNo = r;
        name = n;
    }

    void display() {
        cout << "Roll No: " << rollNo << ", Name: " << name << endl;
    }
};

int main() {
    Student s1;               // Calls default constructor
    Student s2(101, "Ahsan"); // Calls parameterized constructor

    cout << "Default Constructor:\n";
    s1.display();

    cout << "Parameterized Constructor:\n";
    s2.display();

    return 0;
}
```

Task 1: Default Constructor — Employee Management System

Create a class named Employee for a basic employee management system. The class should have the following private data members:

- id (int)
- name (string)
- salary (float)

Implement a default constructor that initializes:

- id = 0
- name = "Not Assigned“
- salary = 0.0

Also include:

- A displayDetails() method that shows employee details.
- A main() function that creates an object using the default constructor and displays its information.

Objective: Simulate how a new employee might be stored in the system before their details are updated.

Task 2: Parameterized Constructor — Bank Account Initialization

Create a class named BankAccount that manages account information. It should have the following private data members::

- accountNumber (string)
- accountHolder (string)
- balance (double)

Implement a parameterized constructor to initialize all three values when a new account is created:

- accountNumber - PK0309800780100
- accountHolder - Muhammad Awais
- balance - 10000

Also include:

- A method showAccountDetails() to display the account information..
- In main(), create an object of BankAccount using user-defined values and display the account details..

Objective: Demonstrate how real banking systems initialize customer data upon account creation.

Task 3: Constructor Overloading

Create a class named Rectangle with the following private data members:

- length (float)
- width (float)

Overload the constructor as follows:

- A default constructor that initializes both length and width to 1.0.
- A parameterized constructor that takes two float values to initialize length and width.
- A single-parameter constructor that sets both length and width to the same value (creating a square).

Also include:

- A method area() to return the area of the rectangle and a display() method.
- In main(), create three objects using all constructor versions and display their dimensions and area.

Objective: Demonstrate how a various version of the constructors can be created using overloading.

Destructor in C++

- A special member function that is automatically called when an object goes out of scope.
- Used to release resources (memory, files, network, etc.).
- Name is same as class, prefixed with a tilde (~)

```
class MyClass {
public:
    ~MyClass() {
        // Cleanup code here
    }
};
```

Key Points

- Called automatically for stack objects.
- Only one destructor per class (no overloading).
- No parameters, no return type.
- If not defined, C++ provides a default destructor

When is Destructor Called?

- When the object goes out of scope (stack).
- When you explicitly use delete (heap)

```
{
    MyClass obj;     // Destructor called automatically
}
MyClass* ptr = new MyClass();
delete ptr;         // Destructor called manually
```

When Should You Define a Destructor?

- You allocate memory with new.
- You open files or use system/network resources.
- You want to track when objects are destroyed (e.g., for logging/debugging).

Practice Question

- What will be the output of the following code?

```
class MyClass {
public:
    MyClass() { cout << "Constructor\n"; }
    ~MyClass() { cout << "Destructor\n"; }
};

int main() {
    MyClass obj;
    return 0;
}
```

Output

```
Constructor
Destructor
```

Resource Cleanup Example

```
class MyArray {
private:
    int* data;
public:
    MyArray(int size) {
        data = new int[size];
        std::cout << "Array of size " << size << " created.\n";
    }

    ~MyArray() {
        delete[] data;
        std::cout << "Array destroyed and memory released.\n";
    }
};
```

Task 4: Destructor

Create a class called Locker that represents a bank locker.

- The constructor should print:

  "Locker allocated to customer."

- The destructor should print:

  "Locker returned by customer."

Inside your main() function:

- Create one locker object inside a block { } to observe automatic destructor call.
- Create another locker using new and release it using delete.

Objective:

- Understand when destructors are called.
- Practice creating objects on stack and heap.
- Learn the importance of managing memory and cleanup.

REFERENCES

- https://www.w3schools.com/cpp/cpp_constructors.asp